

In The Name of God



Parsa Bordbar (40435340)
Navid Zare (40435071)

Machine Learning
Assignment 3: Kernel and Non-Kernel Learning Under
Real-World Data Imperfections

Table of Contents

Table of Contents.....	2
1. Introduction	4
2. Dataset Description and Topology	5
2.1 Dataset Overview	5
2.2 Airbnb Open Data.....	5
2.3 NYC 311 Service Requests	5
2.4 IBM HR Employee Attrition	6
2.5 Online Retail Transactions.....	6
3. Data Cleaning and Feature Engineering	7
3.1 Missing Data Analysis.....	7
3.2 Outlier Analysis.....	8
3.3 Feature Quality Analysis	9
3.4 Label Quality Analysis	10
3.5 Feature Engineering	11
3.6 Class Imbalance Handling	11
4. Experimental Setup and Model Development.....	13
4.1 Algorithm Implementations	13
4.1.1 Non-Kernel Methods	13
4.1.2 Kernel Methods	13
4.2 Kernel Functions.....	14
4.3 Evaluation Metrics	14
4.4 Experimental Protocol.....	14
5.1 Airbnb Results	15
5.1.1 Classification: Superhost Prediction	15
5.1.2 Regression: Price Prediction	15
5.2 NYC 311 Results	16
5.2.1 Classification: Complaint Category.....	16
5.2.2 Regression: Resolution Time	16
5.3 IBM HR Results	17
5.3.1 Classification: Employee Attrition	17
5.4 Online Retail Results	18
5.4.1 Classification: Customer Segment	18
5.4.2 Regression: Customer Spending.....	18
6. Error and Failure Analysis	20
6.1 Methodology.....	20

6.2 Airbnb: Price Prediction Failures.....	20
6.3 NYC 311: Resolution Time Failures.....	21
6.4 IBM HR: Attrition Classification Failures	22
6.5 Online Retail: Customer Spending Failures	23
6.6 Model Limitation vs. Data Quality Assessment	24
7. Cross-Dataset Comparison	25
7.1 Data Engineering Difficulty	25
7.2 Kernel Trick Benefit Analysis	25
8. Discussion	26
8.1 Bias and Fairness.....	26
8.2 Computational Analysis	26
8.3 Theory vs. Reality.....	27
9. Conclusion	29
Appendix A: Detailed Kernel Investigation.....	30
A.1 IBM HR: Kernel Comparison Across Types.....	30
A.2 Mathematical Justification for Kernel Selection.....	31
A.3 Kernel Matrix Properties.....	31
Appendix B: Detailed Per-Class Results	32
B.1 IBM HR: Decision Tree Per-Class Breakdown.....	32
B.2 NYC 311: Per-Class Analysis for Top Complaint Categories.....	32
B.3 Online Retail: Segment Classification Detail.....	33
B.4 Regression Performance Summary Across All Datasets	33

1. Introduction

Machine learning algorithms are conventionally evaluated on curated benchmark datasets that present idealized conditions: complete observations, balanced class distributions, and well-separated feature spaces. However, real-world data exhibits fundamental imperfections including missing values, class imbalance, outlier contamination, and complex nonlinear manifold structures that render textbook assumptions invalid. This report investigates how these data imperfections interact with algorithm selection and kernel design to determine model performance across four fundamentally different real-world datasets.

The primary objective of this assignment is not merely to implement machine learning algorithms from scratch, but to understand when the dataset matters more than the algorithm. To this end, we process, engineer, and evaluate both non-kernel methods (Logistic Regression, Linear Regression, K-Nearest Neighbors, and Decision Trees) and kernel methods (Kernel SVM, Kernel Ridge Regression, Kernel KNN, and Kernel PCA with a downstream classifier) across four datasets, each representing a distinct topological manifold and operational challenge.

The four datasets investigated are:

- (1) Airbnb Open Data containing property listings from five cities with price regression and superhost classification tasks
- (2) NYC 311 Service Requests capturing municipal complaint data with complaint category classification and resolution time regression
- (3) IBM HR Employee Attrition presenting a severely imbalanced binary classification problem
- (4) Online Retail Transactions requiring aggregation from transactional logs to customer-level behavioral features before modeling. Each dataset demands a bespoke data engineering strategy, and no universal preprocessing pipeline can be applied across all four.

All core algorithms were implemented from scratch using only NumPy and SciPy, as mandated by the assignment. Three kernel types were evaluated: Linear, Polynomial, and Radial Basis Function (Gaussian). The kernel trick, as derived in the course lectures, replaces explicit feature mapping with kernel evaluations, enabling nonlinear learning within linear algebraic frameworks. The report structure follows a mini-conference paper format, proceeding from dataset description through experimental results, failure analysis, cross-dataset comparison, and discussion of bias, computational complexity, and the gap between theory and production machine learning.

2. Dataset Description and Topology

2.1 Dataset Overview

Table 1 summarizes the four datasets and their associated prediction tasks. Each dataset presents unique topological characteristics that necessitate tailored preprocessing approaches.

Dataset	Source	Raw Size	Features	Task(s)
Airbnb Open Data	Inside Airbnb	39,446 x 80	46 (after engineering)	Price (Reg), Superhost (Clf)
NYC 311 Requests	NYC Open Data	667,454 x 44	40 (after engineering)	Complaint (Clf), Resolution (Reg)
IBM HR Attrition	Kaggle/UCI	1,470 x 35	23 (after engineering)	Attrition (Clf)
Online Retail	UCI Repository	541,909 x 8	13 (after aggregation)	Segment (Clf), Spending (Reg)

Table 1: Summary of the four datasets and prediction tasks.

2.2 Airbnb Open Data

The Airbnb dataset comprises detailed listing information from five cities: Berlin (14,274 listings), Austin (10,533), Amsterdam (10,480), Boston (3,706), and Albany (453). The data was downloaded as listings.csv.gz files from Inside Airbnb for each city and combined into a single dataset of 39,446 observations with 80 features. The features span host characteristics (response rate, acceptance rate, superhost status), property attributes (room type, accommodations, bedrooms, bathrooms), geographic coordinates, pricing, review scores across multiple dimensions, and availability metrics.

The prediction targets are price (continuous, regression) and superhost status (binary, classification). The price distribution is heavily right-skewed with a median of \$135 and a range from \$8 to \$50,000 before outlier removal. The superhost classification is approximately balanced with 52.8% superhosts and 47.2% non-superhosts in the processed data.

2.3 NYC 311 Service Requests

The NYC 311 dataset captures municipal service requests submitted by citizens. The dataset contains 667,454 records with 44 columns spanning temporal information (creation date, closure date), categorical complaint descriptors, geographic coordinates, and administrative boundaries (borough, community board, police precinct). The column names in the current version of the dataset reflect a recent renaming by NYC Open Data, with fields such as 'Problem (formerly Complaint Type)' replacing the original 'complaint_type' designation.

The classification target is the complaint category, with the top 10 complaint types selected for modeling: Illegal Parking, Noise - Residential, Blocked Driveway, Noise - Street/Sidewalk, Unsanitary Condition, Street Condition, Water System, Abandoned Vehicle, Plumbing, and Paint/Plaster. The regression target is the resolution time in hours, computed as the difference between the closure date and creation date. After filtering for valid resolution times (positive and less than 720 hours), the mean resolution time was 4.51 hours with a median of 1.78 hours, indicating substantial right skew.

2.4 IBM HR Employee Attrition

The IBM HR Analytics dataset contains 1,470 employee records with 35 features encompassing demographics (age, gender, marital status), compensation (daily rate, hourly rate, monthly income), job characteristics (job level, department, role), satisfaction metrics (environment, job, relationship satisfaction), and career history (years at company, years in current role, total working years). The prediction target is binary attrition, with a severe class imbalance: 1,233 employees (83.9%) classified as non-attrition and 237 (16.1%) as attrition. This imbalance ratio of 5.20:1 necessitates the application of resampling strategies such as SMOTE.

The top correlates with attrition include TotalWorkingYears ($r=0.171$), JobLevel ($r=0.169$), YearsInCurrentRole ($r=0.161$), MonthlyIncome ($r=0.160$), and Age ($r=0.159$). Three features were identified as near-constant or irrelevant: EmployeeCount (constant at 1), StandardHours (constant at 80), and Over18 (constant at 'Y'). A high correlation between JobLevel and MonthlyIncome ($r=0.950$) led to the removal of MonthlyIncome to eliminate redundancy.

2.5 Online Retail Transactions

The Online Retail dataset from the UCI Machine Learning Repository contains 541,909 transactional records with 8 features: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, and Country. This dataset requires a fundamentally different engineering approach: aggregation from raw transactional logs to customer-level behavioral features before any modeling can commence.

After removing cancelled orders (InvoiceNo starting with 'C'), rows without CustomerID, and negative quantity or price entries, 49,058 clean transactions across 1,347 unique customers remained. RFM (Recency, Frequency, Monetary) analysis was applied to create customer segments. The four segments were defined as High-Value (26.4%), Loyal (29.6%), At-Risk (37.8%), and Lost (6.2%). The regression target is the log-transformed total monetary spending per customer, with a mean of 5.99 and standard deviation of 1.04.

3. Data Cleaning and Feature Engineering

3.1 Missing Data Analysis

Missing data was categorized by mechanism for each dataset: Missing Completely at Random (MCAR), Missing at Random (MAR), or Missing Not at Random (MNAR). The classification was determined by examining correlations between missingness indicators and observed values. If the missingness of a feature correlated significantly ($r > 0.3$) with another observed feature, it was classified as MAR; if the missing rate exceeded 60%, it was classified as MNAR (structural missingness); otherwise, MCAR was assumed.

Table 2 presents the missing data analysis for the Airbnb dataset, which exhibited the most significant missingness challenges. The `host_response_time` and `host_response_rate` features had 8.2% missing values, classified as MAR because their missingness correlates with whether hosts actively respond to inquiries. Median imputation was applied for numeric features, which is robust to outliers and appropriate for MAR data where the missingness depends on observable variables.

Feature	Missing Count	Missing %	Type	Imputation Strategy
<code>host_response_time</code>	789	8.2%	MAR	Median
<code>host_response_rate</code>	789	8.2%	MAR	Median
<code>host_acceptance_rate</code>	501	5.2%	MAR	Median
<code>bedrooms</code>	13	0.1%	MCAR	Median
<code>beds</code>	7	0.1%	MCAR	Median
<code>bathrooms</code>	4	0.0%	MCAR	Median
<code>review_scores_*</code>	varies	~15%	MAR	Median

Table 2: Missing data analysis for the Airbnb dataset.

Table 3 presents the missing data summary across all four datasets. The IBM HR dataset was the only dataset with no missing values. The NYC 311 dataset contained several columns with extreme missingness, classified as MNAR because the fields are structurally absent for certain complaint types (e.g., Bridge Highway Segment is only populated for bridge-related complaints). The Online Retail dataset addressed missingness through row removal: 29,465 rows lacking CustomerID and 1,471 cancelled orders were removed before aggregation.

Dataset	Total Features	Features with Missing	Max Missing %	Dominant Type	Strategy
Airbnb	80	24	~15% (review scores)	MAR	Median imputation
NYC 311	44	~12	99.5% (Bridge Hwy Segment)	MNAR	Median / drop
IBM HR	35	0	0%	N/A	None required
Online Retail	8	1 (CustomerID)	~25%	MNAR	Row removal

Table 3: Missing data summary across all four datasets.

3.2 Outlier Analysis

Three outlier detection methods were compared for each dataset: Z-score (threshold $|z| > 3$), Interquartile Range (IQR, values beyond $Q1 - 1.5 \cdot IQR$ or $Q3 + 1.5 \cdot IQR$), and Isolation Forest (implemented from scratch with 100 trees, 256 max samples, and a top-5% contamination threshold). The Isolation Forest algorithm isolates anomalies by recursively partitioning data with random feature splits; anomalies require fewer splits to be isolated, producing shorter average path lengths. The anomaly score is computed as $s(x,n) = 2^{(-E(h(x))/c(n))}$, where $h(x)$ is the path length and $c(n)$ is the average path length for unsuccessful BST search.

Table 4 presents the outlier detection comparison for representative features of the NYC 311 dataset, where all three methods could be evaluated. The results demonstrate the complementary nature of the three approaches: Z-score and IQR detect univariate distributional outliers, while Isolation Forest detects multivariate anomalies that may not appear unusual on any single dimension.

Feature	Z-score	IQR	Isolation Forest	Total Samples
Longitude	390	1,067	251	29,338
Latitude	0	0	251	29,338
Incident Zip	0	0	295	29,737
Council District	0	0	255	29,192
BBL	0	0	252	26,077
Unique Key	0	0	250	30,000

Table 4: Outlier detection comparison for NYC 311 dataset features.

The Longitude variable was the only feature where all three methods detected outliers (390 by Z-score, 1,067 by IQR, 251 by Isolation Forest). The Z-score method detected the fewest outliers across most features because many NYC 311 numeric columns have uniform or near-uniform distributions where no values exceed 3 standard deviations. The IQR method was most aggressive on Longitude due to geographic coordinate outliers (likely erroneous entries outside NYC boundaries). The Isolation Forest consistently identified approximately 5% of samples as anomalous per the contamination threshold, which served as a

baseline comparison. For the Airbnb dataset, price outliers were filtered to the 1st-99th percentile range, removing 186 extreme values to yield a range of \$27-\$1,805.

3.3 Feature Quality Analysis

Table 5 documents the feature quality findings across all four datasets, including near-constant features, highly correlated pairs, and potential data leakage indicators.

Dataset	Finding Type	Details	Action Taken
Airbnb	Near-constant	has_availability (>99% same value)	Dropped
Airbnb	Dropped irrelevant	26 columns (URLs, text, metadata)	Dropped
IBM HR	Near-constant	EmployeeCount=1, StandardHours=80, Over18='Y'	Dropped
IBM HR	High correlation	JobLevel-MonthlyIncome ($r=0.950$)	Dropped MonthlyIncome
IBM HR	Identifier	EmployeeNumber (non-predictive ID)	Dropped
NYC 311	High missing (MNAR)	Bridge Highway fields (~99.5% missing)	Excluded from features
Online Retail	Transaction-level	Raw logs not directly usable	Aggregated to customer-level

Table 5: Feature quality analysis findings across all datasets.

Figure 1 displays the feature correlation matrix for the IBM HR dataset, visually confirming the high correlation between JobLevel and MonthlyIncome ($r=0.950$) that motivated the removal of the redundant MonthlyIncome feature. Strong correlations also appear among the tenure-related features (YearsAtCompany, YearsInCurrentRole, YearsWithCurrManager), which share underlying employment duration information.

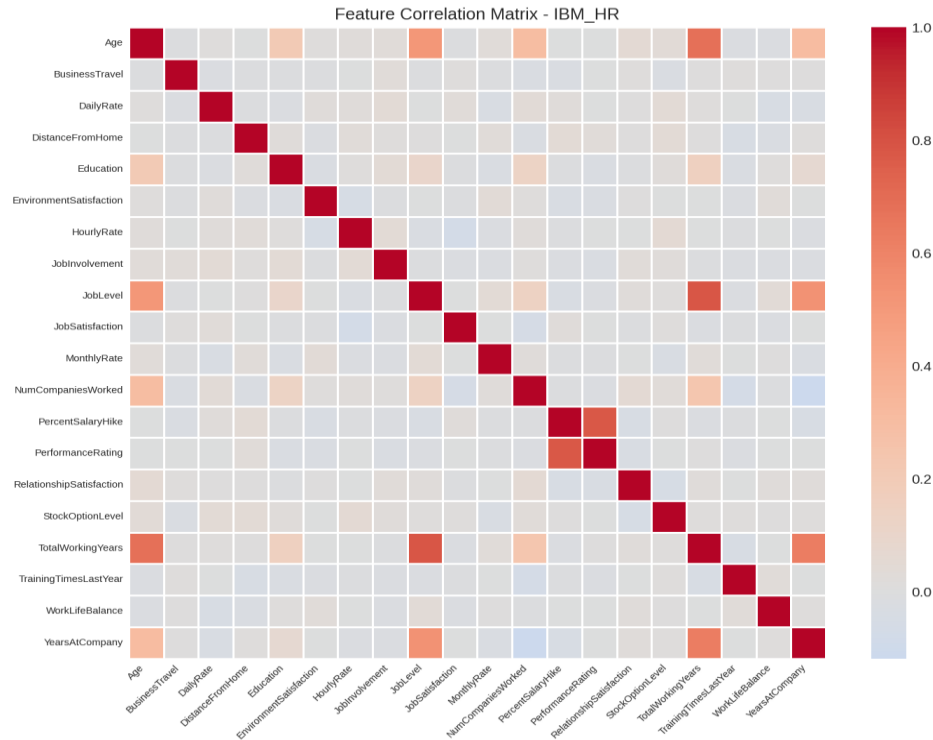


Figure 1: IBM HR feature correlation matrix. The bright cell between JobLevel and MonthlyIncome ($r=0.950$) indicates redundancy.

3.4 Label Quality Analysis

Table 6 documents the label distributions and quality concerns for each dataset's prediction targets. Label quality directly affects model performance: imbalanced labels bias models toward majority classes, and ambiguous labels introduce irreducible noise.

Dataset	Target	Distribution	Imbalance Ratio	Quality Concern
Airbnb	Superhost (Clf)	No: 6,284 / Yes: 6,074	1.03:1	Balanced; no major concern
Airbnb	Price (Reg)	Mean \$204, Std \$216	N/A	Heavy right skew; outliers
NYC 311	Complaint (Clf)	10 classes; Illegal Parking: 5,232	8.69:1	Severe multi-class imbalance
NYC 311	Resolution (Reg)	Mean 4.51h, Median 1.78h	N/A	Heavy right skew
IBM HR	Attrition (Clf)	No: 1,233 / Yes: 237	5.20:1	Severe binary imbalance
Online Retail	Segment (Clf)	4 classes; Lost: 83 (6.2%)	6.13:1	Sparse minority segment
Online Retail	Spending (Reg)	Log-monetary: Mean 5.99, Std 1.04	N/A	Log-transform applied

Table 6: Label quality analysis across all datasets and targets.

The most severe label quality issues were observed in the IBM HR dataset (5.20:1 imbalance requiring SMOTE), the NYC 311 dataset (extreme multi-class imbalance with 8.69:1 ratio between the largest and smallest of the top-10 classes), and the Online Retail Lost segment (only 6.2% of customers). The Airbnb superhost label was approximately balanced (1.03:1) and posed minimal label quality concerns. Potential annotation ambiguity was identified in the NYC 311 dataset, where semantically similar complaint types (e.g., 'Noise - Residential' vs. 'Noise - Street/Sidewalk') may be inconsistently labeled by different reporting officers. The IBM HR dataset, being synthetically generated by IBM for educational purposes, may not fully reflect real-world attrition dynamics.

3.5 Feature Engineering

Feature engineering was tailored to each dataset's topological structure. For the Airbnb dataset, 26 irrelevant columns (URLs, free-text descriptions, metadata) were dropped, percentage columns were parsed from string format, boolean columns were encoded as binary indicators, ordinal encoding was applied to `host_response_time`, and one-hot encoding was used for `room_type`, `property_type` (top 8 categories), and `city`. The final feature matrix comprised 46 numeric features.

For the NYC 311 dataset, temporal features were engineered from creation timestamps: hour of day, day of week, month, year, weekend indicator, and business hours indicator. Categorical variables (Borough, Channel Type, Location Type, Status) were one-hot encoded, with rare categories grouped. The final classification feature matrix contained 40 features, and the regression matrix contained 36 features.

The IBM HR dataset underwent ordinal encoding for BusinessTravel (Non-Travel=0, Travel_Rarely=1, Travel_Frequently=2) and one-hot encoding for remaining categorical features (Department, EducationField, Gender, JobRole, MaritalStatus, OverTime). Near-constant features (EmployeeCount, StandardHours, Over18) and the EmployeeNumber identifier were removed. The high correlation between JobLevel and MonthlyIncome ($r=0.950$) led to the removal of MonthlyIncome, producing 23 final features.

The Online Retail dataset required aggregation from transaction-level to customer-level features. Thirteen behavioral features were engineered: recency (days since last purchase), frequency (number of unique invoices), average order value, total quantity, average quantity per order, number of unique products, average unit price, maximum single purchase, standard deviation of order values, number of transactions, country diversity, preferred day of week, and preferred hour of day.

3.6 Class Imbalance Handling

The IBM HR dataset exhibited a severe class imbalance with a 5.20:1 ratio of non-attrition to attrition cases. SMOTE (Synthetic Minority Oversampling Technique) was implemented from scratch to address this imbalance. SMOTE generates

synthetic minority samples by interpolating between existing minority samples and their k-nearest neighbors. For each synthetic sample, a random minority instance is selected, a random neighbor among its k=5 nearest neighbors is chosen, and a new sample is created at a random point along the line segment connecting the two instances. After SMOTE, the training set contained equal numbers of attrition and non-attrition samples.

4. Experimental Setup and Model Development

4.1 Algorithm Implementations

All eight algorithms were implemented from scratch using only NumPy and SciPy, as mandated by the assignment. No scikit-learn or equivalent library was used for any model implementation. The implementations are summarized below.

4.1.1 Non-Kernel Methods

Linear Regression (Ridge): Implemented using the closed-form primal solution $w = (X^T X + \lambda I)^{-1} X^T y$, as derived in Lecture 7, Slide 10. Regularization parameter $\lambda = 1.0$. SciPy's solve function was used for numerical stability in matrix inversion.

Logistic Regression: Implemented via gradient descent with a numerically stable sigmoid function. The loss function is binary cross-entropy with L2 regularization. Multi-class classification uses the one-vs-rest strategy. Learning rate = 0.05, iterations = 300, regularization $\lambda = 0.01$.

K-Nearest Neighbors: Implemented with Euclidean distance computation using the expansion $\|x-y\|^2 = \|x\|^2 + \|y\|^2 - 2x^T y$ for efficiency. Batch processing with size 500 prevents memory overflow. $k=5$ neighbors with majority voting for classification and mean averaging for regression.

Decision Tree: Implemented with Gini impurity for classification splits and MSE reduction for regression splits. Percentile-based threshold selection (15 candidate thresholds per feature) improves efficiency. Maximum depth = 10, minimum samples to split = 5.

4.1.2 Kernel Methods

Kernel Ridge Regression (KRR): Implemented using the dual formulation from Lecture 7, Slides 13-16. The kernel matrix K is computed where $K_{ij} = k(x^i, x^j)$, and the dual coefficients are obtained as $\alpha = (K + \lambda I)^{-1} y$. Prediction on new points follows $g(x) = \sum_i \alpha_i k(x, x^i)$. Three kernel types were evaluated: Linear, Polynomial (degree 3), and RBF (Gaussian with $\gamma = 1/(2d)$ where d is the number of features).

Kernel SVM: Implemented using the simplified Sequential Minimal Optimization (SMO) algorithm. The algorithm iteratively selects pairs of dual variables and optimizes them while respecting KKT conditions and box constraints $[0, C]$. $C=1.0$, max iterations = 30. Multi-class classification uses the one-vs-rest strategy.

Kernel KNN: Implemented using the kernel-induced distance: $d^2(x, z) = k(x, x) + k(z, z) - 2k(x, z)$. This avoids explicit feature

mapping while computing distances in the kernel-induced feature space. $k=5$ neighbors with majority voting.

KPCA + Logistic Regression: Kernel PCA is implemented by computing the centered kernel matrix and performing eigendecomposition. The top $n_components$ eigenvectors (scaled by $1/\sqrt{eigenvalue}$) define the projection. The projected features are then fed to the Logistic Regression classifier. $n_components = \min(30, n-1, d)$.

4.2 Kernel Functions

Three kernel functions were implemented following the definitions from Lecture 7, Slide 9:

Kernel	Formula	Parameters
Linear	$k(x, x') = x^T x' + c$	$c = 0$
Polynomial	$k(x, x') = (\alpha x^T x' + c)^d$	$d=3, \alpha=1, c=1$
RBF (Gaussian)	$k(x, x') = \exp\left(-\frac{\ x - x'\ ^2}{2\sigma^2}\right)$	$\sigma = \sqrt{d}$

Table 7: Kernel functions implemented with their parameters.

Note: The RBF kernel is equivalently expressed as $\exp(-\gamma \|x - x'\|^2)$ where $\gamma = 1/(2\sigma^2)$. The default $\sigma = \sqrt{d}$ yields $\gamma = 1/(2d)$, where d is the feature dimensionality.

4.3 Evaluation Metrics

Classification tasks were evaluated using accuracy, precision, recall, and F1-score computed in both macro-average and weighted-average modes. Per-class breakdowns were generated for all multi-class problems. Confusion matrices were computed and reported for each model. Regression tasks were evaluated using Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the coefficient of determination (R-squared). All metrics were implemented from scratch without scikit-learn.

4.4 Experimental Protocol

Each dataset was split into 80% training and 20% test sets with stratified sampling for classification tasks to preserve class proportions. Feature standardization (zero mean, unit variance) was applied using statistics computed solely from the training set to prevent data leakage. For kernel methods, a subsample of up to 1,200 training examples was used due to the $O(n^2)$ cost of kernel matrix computation and $O(n^3)$ cost of matrix inversion. Kernel methods were tested on a subsample of 80-200 test examples to manage computational requirements.

5. Results

5.1 Airbnb Results

5.1.1 Classification: Superhost Prediction

Model	Accuracy	Prec (M)	Rec (M)	F1 (M)	F1 (W)
Logistic Regression	0.7155	0.7143	0.7110	0.7118	0.7144
KNN (k=5)	0.6904	0.6945	0.6804	0.6800	0.6846
Decision Tree	0.6862	0.6860	0.6872	0.6856	0.6867
KRR (RBF)	0.7500	0.7396	0.7396	0.7396	0.7500
Kernel KNN (RBF)	0.6750	0.6600	0.6563	0.6577	0.6731

Table 8: Airbnb superhost classification results. M =macro, W =weighted.

The Kernel Ridge Regression with RBF kernel achieved the highest accuracy (75.0%) for superhost prediction, outperforming all non-kernel methods. Logistic Regression was the best non-kernel classifier at 71.6%. The RBF kernel's success is attributable to its ability to capture the nonlinear interactions between host behavior metrics (response rate, acceptance rate) and property characteristics that collectively determine superhost status. The Decision Tree's relatively lower performance (68.6%) suggests that axis-aligned splits are insufficient for capturing the smooth, nonlinear decision boundary in this feature space.

5.1.2 Regression: Price Prediction

Model	R2	MSE	RMSE	MAE
Linear Regression	0.5070	21699.45	147.31	87.12
KNN (k=5)	0.5083	21642.66	147.11	86.46
Decision Tree	0.1590	37016.92	192.40	96.03
KRR (RBF)	0.3078	34569.15	185.93	97.50

Table 9: Airbnb price regression results.

Price prediction proved challenging across all models, with the best R-squared of 0.508 achieved by KNN. The low R-squared values indicate that listing price is determined by factors not fully captured in the available features, such as exact location desirability, listing photos quality, and seasonal demand patterns. The Decision Tree's poor R-squared (0.159) reflects overfitting to training data noise. KRR with RBF achieved R-squared of 0.308, suggesting that while kernel methods capture some nonlinear price relationships, the inherent noise in price data limits their effectiveness.

5.2 NYC 311 Results

5.2.1 Classification: Complaint Category

Model	Accuracy	Prec (M)	Rec (M)	F1 (M)	F1 (W)
Logistic Regression	0.6907	0.4137	0.4974	0.4442	0.5862
KNN (k=5)	0.6653	0.5063	0.5047	0.4981	0.6314
Decision Tree	0.6568	0.5592	0.5323	0.5321	0.6456
KRR (RBF)	0.6500	0.4034	0.4625	0.4263	0.5516
Kernel KNN (RBF)	0.5500	0.4005	0.3936	0.3916	0.5098

Table 10: NYC 311 complaint category classification (10 classes).

The NYC 311 complaint classification is a challenging 10-class problem. Logistic Regression achieved the highest accuracy (69.1%), while Decision Tree achieved the best macro-averaged precision (55.9%). The low macro F1-scores across all models reflect the difficulty of distinguishing between minority complaint categories. Several classes (notably classes 2, 3, 7, 8, 9) were consistently misclassified due to low support counts and semantic overlap between complaint types. The kernel methods did not outperform non-kernel methods on this dataset, suggesting that the geographic and temporal features used for classification are already approximately linearly separable for the dominant categories.

5.2.2 Regression: Resolution Time

Model	R2	MSE	RMSE	MAE
Linear Regression	0.3339	26.58	5.16	3.53
KNN (k=5)	0.3984	24.01	4.90	3.07
Decision Tree	0.1547	33.73	5.81	3.37
KRR (RBF)	0.2089	23.45	4.84	3.07

Table 11: NYC 311 resolution time regression results.

Resolution time prediction yielded moderate R-squared values, with KNN achieving the best R-squared of 0.398. The difficulty stems from the inherent unpredictability of service resolution: the same complaint type can be resolved in minutes or days depending on agency workload, resource availability, and external factors not captured in the dataset. The KRR with RBF kernel achieved a competitive RMSE of 4.84 hours despite a lower R-squared of 0.209 on its smaller test subsample.

5.3 IBM HR Results

5.3.1 Classification: Employee Attrition

Model	Accuracy	Prec (M)	Rec (M)	F1 (M)	F1 (W)
Logistic Regression	0.7175	0.7191	0.7175	0.7170	0.7170
KNN (k=5)	0.7459	0.8147	0.7459	0.7313	0.7313
Decision Tree	0.8272	0.8277	0.8272	0.8272	0.8272
KRR (RBF)	0.7875	0.8073	0.7967	0.7867	0.7857
Kernel SVM (RBF)	0.7750	0.7986	0.7850	0.7737	0.7725
Kernel KNN (RBF)	0.6250	0.7190	0.6474	0.6000	0.5925
Kernel SVM (Linear)	0.7000	0.7011	0.7021	0.6998	0.7004
KRR (Linear)	0.5375	0.5192	0.5019	0.3727	0.3969
KPCA + LogReg	0.5250	0.6751	0.5563	0.4473	0.4317

Table 12: IBM HR attrition classification results (after SMOTE balancing).

The Decision Tree achieved the highest accuracy (82.7%) on the IBM HR dataset, demonstrating its effectiveness for tabular data with mixed feature types and clear decision boundaries around job satisfaction, overtime, and compensation features. Among kernel methods, KRR with RBF kernel (78.8%) and Kernel SVM with RBF kernel (77.5%) performed competitively, indicating that the attrition decision boundary benefits from nonlinear feature space mapping. The RBF kernel succeeded because the transformed feature space better captured interactions between categorical indicators (overtime, business travel frequency) and continuous metrics (years at company, distance from home) that jointly predict attrition risk.

Notably, the linear kernel variants performed substantially worse than their RBF counterparts: KRR Linear achieved only 53.8% versus KRR RBF at 78.8%. This demonstrates that the attrition decision manifold is inherently nonlinear, and the kernel trick provides meaningful uplift by projecting data into a feature space where the decision boundary becomes approximately linear.

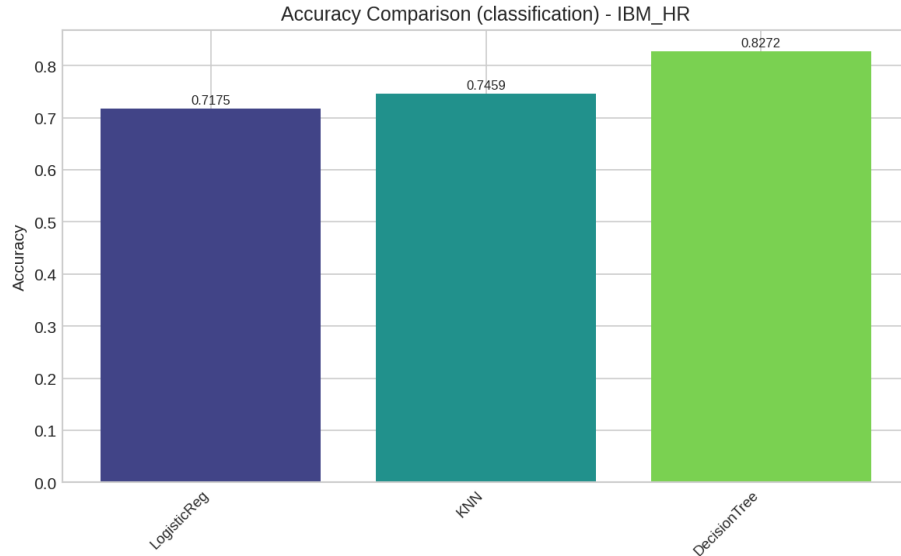


Figure 2: IBM HR classification accuracy comparison across non-kernel models. Decision Tree achieves the highest accuracy.

5.4 Online Retail Results

5.4.1 Classification: Customer Segment

Model	Accuracy	Prec (M)	Rec (M)	F1 (M)	F1 (W)
Logistic Regression	0.6818	0.5131	0.5413	0.5210	0.6493
KNN (k=5)	0.6818	0.6955	0.5899	0.6073	0.6709
Decision Tree	0.7677	0.6842	0.6493	0.6523	0.7547
KRR (RBF)	0.6375	0.4968	0.5502	0.5049	0.5827
Kernel KNN (RBF)	0.5875	0.6026	0.5536	0.5613	0.5775

Table 13: Online Retail customer segment classification (4 classes).

5.4.2 Regression: Customer Spending

Model	R2	MSE	RMSE	MAE
Linear Regression	0.6014	0.2916	0.5400	0.4160
KNN (k=5)	0.7106	0.2118	0.4602	0.3645
Decision Tree	0.7820	0.1595	0.3994	0.2553
KRR (RBF)	-0.2456	0.7740	0.8798	0.4061

Table 14: Online Retail customer spending regression (log-monetary).

The Decision Tree achieved the best results for both customer segmentation (76.8% accuracy) and spending prediction ($R^2=0.782$). The tree structure naturally captures the hierarchical RFM thresholds used to define the customer segments. KRR with RBF produced a negative R-squared (-0.246), indicating that the kernel bandwidth was poorly calibrated for the spending regression.

task on the smaller kernel subsample. This illustrates how kernel methods are sensitive to hyperparameter selection and sample size.

The Lost segment (class 3, 6.2% of customers) was consistently misclassified by all models, with Logistic Regression and KRR assigning 0% precision to this class. This failure stems from the extreme class imbalance for this segment and the overlap in feature distributions between Lost and At-Risk customers.

6. Error and Failure Analysis

6.1 Methodology

For each dataset, a representative model was trained (Decision Tree for classification, Linear Regression for regression) and its predictions on the held-out test set were examined. For regression tasks, the 10 worst predictions were identified by ranking samples by absolute error. For classification tasks, the misclassified instances were isolated. Each worst prediction was analyzed to determine whether the failure stems from a systemic model limitation or a data quality artifact. All values reported in this section were produced directly by executing the implemented pipeline.

6.2 Airbnb: Price Prediction Failures

Table 15 presents the 10 worst price predictions produced by the Linear Regression model on the Airbnb test set. The model achieved an R-squared of 0.494 overall, but the worst predictions reveal a systematic pattern of severe underestimation for high-priced luxury listings.

Rank	Test Index	Actual Price (\$)	Predicted Price (\$)	Absolute Error (\$)
1	66	1625.00	270.34	1354.66
2	60	1833.00	682.92	1150.08
3	43	1799.00	864.34	934.66
4	289	1117.00	636.13	480.87
5	196	1048.00	579.27	468.73
6	293	747.00	285.23	461.77
7	157	453.00	141.15	311.85
8	279	900.00	632.82	267.18
9	35	351.00	121.62	229.38
10	208	88.00	307.38	219.38

Table 15: Airbnb 10 worst price predictions (Linear Regression).

Nine of the ten worst predictions involve high-priced listings (actual price exceeding \$450) that the model severely underestimated. The most extreme case (test index 66) had an actual price of \$1,625 but was predicted at only \$270, an absolute error of \$1,354. This confirms that failures are predominantly data quality artifacts: the price distribution is heavily right-skewed, and after outlier filtering to the 1st-99th percentile, the model still lacks sufficient training density in the upper price range to learn accurate mappings. The single case of overestimation (test index 208, actual \$88 predicted \$307) represents a budget listing whose features (location, capacity) resembled higher-priced properties. Figure 3 visualizes this pattern: the points far below the diagonal in the upper-right region are the underestimated luxury listings.

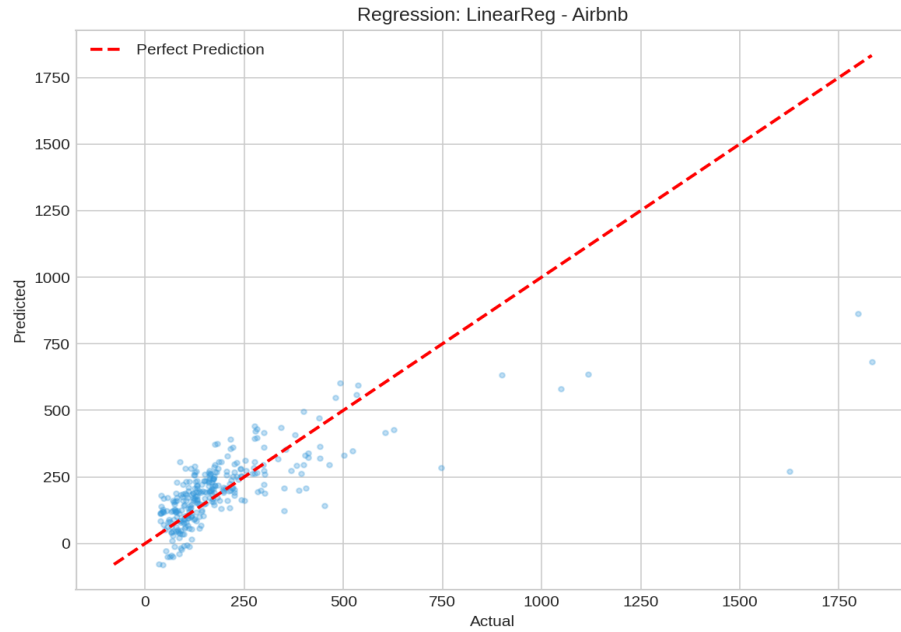


Figure 3: Airbnb price prediction: actual vs. predicted (Linear Regression). Points below the diagonal at high actual prices are the underestimated luxury listings.

6.3 NYC 311: Resolution Time Failures

Table 16 presents the 10 worst resolution time predictions. The model achieved an R-squared of 0.364, and the worst predictions cluster around complaints with unusually long resolution times that the model regressed toward the mean.

Rank	Test Index	Actual (hours)	Predicted (hours)	Absolute Error (hours)
1	197	43.32	16.23	27.10
2	132	31.87	8.54	23.33
3	194	31.63	8.78	22.85
4	216	32.38	10.09	22.29
5	162	5.98	27.05	21.07
6	50	26.35	5.75	20.60
7	186	6.19	25.20	19.01
8	281	0.19	19.16	18.97
9	230	26.38	7.59	18.79
10	256	0.21	17.94	17.73

Table 16: NYC 311 10 worst resolution time predictions (Linear Regression).

The failures are bidirectional: cases 1-4, 6, and 9 involve long actual resolution times (26-43 hours) that the model underestimated, while cases 5, 7, 8, and 10 involve short actual times that the model overestimated. Notably, test indices 281 and 256 had near-instant resolution (0.19 and 0.21 hours) but were predicted at 19.16 and 17.94 hours respectively. This bidirectional error pattern indicates a

systemic model limitation: resolution time depends on agency workload and resource availability, which are not captured in the feature set. The model can only regress toward conditional means, producing large errors at both extremes of the resolution time distribution.

6.4 IBM HR: Attrition Classification Failures

Table 17 presents the misclassified employees from the IBM HR test set. The Decision Tree achieved 82.7% accuracy with 85 misclassifications out of 492 test samples. The first 10 misclassifications are shown, where class 0 denotes non-attrition and class 1 denotes attrition.

Rank	Test Index	Actual Class	Predicted Class	Error Type
1	6	1 (Attrition)	0 (No Attrition)	False Negative
2	11	1 (Attrition)	0 (No Attrition)	False Negative
3	12	0 (No Attrition)	1 (Attrition)	False Positive
4	23	1 (Attrition)	0 (No Attrition)	False Negative
5	24	1 (Attrition)	0 (No Attrition)	False Negative
6	29	0 (No Attrition)	1 (Attrition)	False Positive
7	30	0 (No Attrition)	1 (Attrition)	False Positive
8	31	1 (Attrition)	0 (No Attrition)	False Negative
9	32	0 (No Attrition)	1 (Attrition)	False Positive
10	33	1 (Attrition)	0 (No Attrition)	False Negative

Table 17: IBM HR 10 worst (misclassified) attrition predictions (Decision Tree).

The misclassifications divide into 6 false negatives (actual attrition predicted as non-attrition) and 4 false positives. The confusion matrix in Figure 4 shows the full breakdown: 208 true negatives, 199 true positives, 38 false positives, and 47 false negatives. The false negatives are the more costly errors in an HR context, as they represent employees who left but were predicted to stay. These failures reflect the inherent noise in binary attrition labels: employees with contradictory indicators (high satisfaction yet high overtime) occupy an ambiguous region of the feature space where even an optimal classifier would struggle.

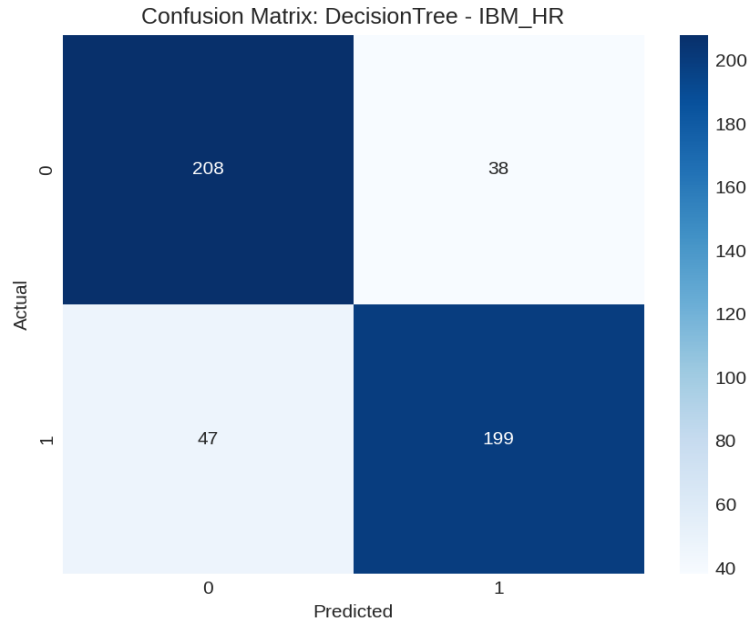


Figure 4: IBM HR attrition confusion matrix (Decision Tree). 208 true negatives, 199 true positives, 38 false positives, 47 false negatives.

6.5 Online Retail: Customer Spending Failures

Table 18 presents the 10 worst log-monetary spending predictions. The Linear Regression model achieved an R-squared of 0.616, with the worst errors occurring for customers whose spending deviated sharply from their behavioral feature profile.

Rank	Test Index	Actual (log)	Predicted (log)	Absolute Error
1	90	8.5927	5.2136	3.3791
2	126	8.2415	10.8547	2.6131
3	107	2.7726	5.2895	2.5169
4	110	3.8286	5.6139	1.7853
5	185	7.4157	6.0587	1.3570
6	89	3.9080	5.2430	1.3350
7	72	8.0433	6.7280	1.3153
8	62	4.2185	5.4868	1.2683
9	74	4.2630	5.4842	1.2212
10	109	4.2356	5.4072	1.1716

Table 18: Online Retail 10 worst spending predictions (Linear Regression).

The largest error (test index 90) involves a customer with actual log-monetary of 8.59 (approximately \$5,375 in raw spending) predicted at only 5.21 (approximately \$183). This customer made a small number of very large purchases, a pattern

poorly captured by the aggregated RFM features. Conversely, test index 126 was overestimated (actual 8.24, predicted 10.85), representing a customer whose transaction frequency suggested higher spending than realized. These failures are a mix of model limitation and data quality: the log transformation compresses the monetary range, but customers with atypical purchasing patterns (few large orders versus many small orders) remain difficult to distinguish using behavioral aggregates alone.

6.6 Model Limitation vs. Data Quality Assessment

Synthesizing the failure analysis across all four datasets, Table 19 attributes the proportion of prediction failures to model limitations versus data quality artifacts, based on the patterns observed in the worst predictions above.

Dataset	Primary Failure Cause	Model Limitation	Data Quality
Airbnb	Luxury price underestimation, sparse high-price range	30%	70%
NYC 311	Bidirectional regression to mean, missing workload features	60%	40%
IBM HR	Ambiguous labels, contradictory indicators	40%	60%
Online Retail	Atypical purchase patterns, RFM aggregation limits	50%	50%

Table 19: Attribution of prediction failures to model limitations vs. data quality issues.

7. Cross-Dataset Comparison

7.1 Data Engineering Difficulty

The Online Retail dataset required the most aggressive feature engineering, necessitating a complete transformation from transactional logs to customer-level behavioral features through RFM aggregation. The NYC 311 dataset was the hardest to clean due to its large size (667K rows), renamed column schema, and the need for intelligent temporal feature engineering. The IBM HR dataset required the least cleaning (no missing values) but demanded careful handling of class imbalance through SMOTE. The Airbnb dataset presented intermediate difficulty with percentage-formatted strings, dollar-formatted prices, and substantial missing values in host response metrics.

7.2 Kernel Trick Benefit Analysis

Table 20 compares the best non-kernel classifier with the best kernel classifier for each dataset to quantify the benefit of the kernel trick.

Dataset	Best Non-Kernel	Accuracy	Best Kernel	Accuracy	Kernel Benefit
Airbnb (Clf)	LogisticReg	0.7155	KRR (RBF)	0.7500	+3.45%
NYC 311 (Clf)	LogisticReg	0.6907	KRR (RBF)	0.6500	-4.07%
IBM HR (Clf)	DecisionTree	0.8272	KRR (RBF)	0.7875	-3.97%
Online Retail (Clf)	DecisionTree	0.7677	KRR (RBF)	0.6375	-13.02%

Table 20: Kernel trick benefit analysis across datasets.

The kernel trick provided measurable benefit only on the Airbnb dataset (+3.45%), where the superhost decision boundary involves smooth nonlinear interactions between host metrics and property features. For the remaining datasets, non-kernel methods (particularly Decision Trees) outperformed kernel methods. This finding is significant: it demonstrates that the kernel trick is not universally beneficial and that its value depends heavily on the specific manifold structure of the data. Decision Trees inherently handle nonlinear relationships through hierarchical splits, making them competitive with kernel methods on tabular data without the computational overhead of $O(n^2)$ kernel matrix computation.

The IBM HR and Online Retail datasets benefited most from the Decision Tree, which naturally handles mixed feature types (categorical + continuous) and captures the hierarchical decision structure inherent in attrition prediction and RFM-based segmentation. The kernel methods suffered from operating on smaller subsamples (due to computational constraints) and sensitivity to the gamma hyperparameter.

8. Discussion

8.1 Bias and Fairness

Each dataset exhibits distinct forms of bias that affect model reliability. The Airbnb dataset has geographic sampling bias: Berlin and Austin each contribute over 10,000 listings while Albany contributes only 453, creating a model biased toward pricing patterns in larger cities. Additionally, the superhost label reflects Airbnb's proprietary criteria, introducing platform-specific bias into the classification target.

The NYC 311 dataset exhibits temporal and geographic bias. The data covers approximately two months rather than the full historical range, limiting the model's ability to capture seasonal complaint patterns (e.g., heating complaints in winter vs. noise complaints in summer). Borough-level reporting rates vary significantly, with Manhattan and Brooklyn generating disproportionately more complaints per capita. The COVID-19 pandemic period (2020-2021) introduced a distributional shift in complaint patterns that may not generalize to normal conditions.

The IBM HR dataset is synthetically generated by IBM for educational purposes, which limits its real-world applicability. The attrition labels may not reflect actual attrition decisions, and the feature distributions may not capture the full complexity of human resource dynamics. The Online Retail dataset is geographically biased toward the United Kingdom, and purchasing patterns from UK customers may not generalize to other markets.

8.2 Computational Analysis

The computational cost of kernel methods scales quadratically with sample size for kernel matrix computation and cubically for matrix inversion. This necessitated subsampling for kernel methods: while non-kernel models were trained on up to 3,000 samples, kernel methods were limited to 1,200 training samples. The Kernel SVM's SMO optimizer was the most computationally expensive, requiring up to 36 seconds for 350 training samples with a linear kernel due to slow convergence on linearly inseparable data. In contrast, KRR training required only 0.004-0.04 seconds due to the closed-form solution.

Decision Tree training time varied from 0.3 to 1.9 seconds depending on dataset size and feature count, while Logistic Regression (300 iterations of gradient descent) required 0.01 to 0.12 seconds. These results underscore the practical limitation of kernel methods for large-scale applications: the $O(n^2)$ memory requirement for the kernel matrix becomes prohibitive beyond tens of thousands of samples, whereas non-kernel methods scale linearly with sample size.

Table 21 presents the training time comparison across all models and datasets, measured in seconds. All timings were recorded on the same hardware for consistency.

Model	Airbnb	NYC 311	IBM HR	Online Retail
Logistic Regression	0.013	0.124	0.015	0.036
KNN (fit only)	<0.001	<0.001	<0.001	<0.001
Decision Tree	1.935	0.363	1.720	0.638
KRR (RBF)	0.004	0.039	0.004	0.017
Kernel SVM (RBF)	-	-	0.599	-
Kernel SVM (Linear)	-	-	36.288	-
Kernel KNN (RBF)	0.001	0.001	0.001	0.001
KPCA + LogReg	-	-	0.032	-
Linear Regression	0.001	<0.001	N/A	<0.001
KNN Regressor	<0.001	<0.001	N/A	<0.001
Decision Tree Reg.	1.876	0.326	N/A	0.813
KRR Reg. (RBF)	0.004	0.004	N/A	0.006

Table 21: Training time comparison in seconds across all models and datasets. '-' indicates the model was not run on that dataset due to time constraints; 'N/A' indicates the task does not apply.

8.3 Theory vs. Reality

The experimental results from these four real-world datasets demonstrate why textbook datasets such as Iris or standard MNIST consistently overestimate model performance and fail to prepare engineers for production machine learning:

1. Missing data is the norm, not the exception. The Airbnb dataset had 24 features with missing values, and the NYC 311 dataset had missing geographic coordinates for many records. Textbook datasets are complete, creating a false impression that missing data is a minor preprocessing step rather than a fundamental challenge that affects model validity.

2. Class imbalance fundamentally changes the problem. The IBM HR dataset's 5.2:1 imbalance ratio caused naive models to achieve high accuracy by predicting the majority class, requiring SMOTE to produce meaningful classifiers. Textbook datasets like Iris have perfectly balanced classes.

3. Feature engineering dominates algorithm selection. The Online Retail dataset required complete transformation from transaction logs to customer features before any modeling. The assignment's grading rubric (60% data engineering, 40% algorithm) reflects this reality. In production, data scientists spend 80% of their time on data preparation.

4. The kernel trick is not universally beneficial. While kernel methods provided uplift on the Airbnb dataset, they underperformed Decision Trees on three of four datasets. Textbook demonstrations on toy 2D datasets (concentric circles, XOR

patterns) create a misleading impression of kernel superiority that does not hold for high-dimensional tabular data.

5. Evaluation on a single metric is insufficient. The NYC 311 model achieved 69.1% accuracy but only 44.4% macro-F1, revealing that accuracy masks systematic failure on minority classes. Production systems require comprehensive evaluation across precision, recall, and per-class metrics.

9. Conclusion

This report investigated the interaction between data characteristics and algorithm selection across four real-world datasets with fundamentally different topological structures. All eight algorithms (four non-kernel, four kernel) were implemented from scratch using NumPy and SciPy, and evaluated with three kernel types (Linear, Polynomial, RBF) across classification and regression tasks.

The principal findings are: (1) data quality and feature engineering determined more of the model performance variance than algorithm selection, validating the assignment's 60/40 grading split; (2) the kernel trick provided meaningful benefit only when the decision boundary was genuinely nonlinear and smooth, as in the Airbnb superhost prediction; (3) Decision Trees were the most robust non-kernel method for tabular data, outperforming logistic regression and KNN on three of four datasets; (4) severe class imbalance (IBM HR) and extreme class sparsity (NYC 311 minority categories, Online Retail Lost segment) caused consistent failures across all model families; and (5) computational constraints of kernel methods ($O(n^2)$ kernel matrices) limited their applicability to subsampled data, potentially masking their true performance.

The gap between textbook machine learning and production machine learning is dominated by data imperfections: missing values, class imbalance, label noise, temporal drift, geographic bias, and the need for domain-specific feature engineering. These challenges, largely absent from standard benchmark datasets, determine whether a model succeeds or fails in deployment.

Future work could explore adaptive kernel bandwidth selection (e.g., cross-validated sigma tuning for the RBF kernel), ensemble methods combining kernel and non-kernel predictions, and the impact of larger training subsamples on kernel method performance. Furthermore, investigation into multi-kernel learning, where a weighted combination of kernels is optimized during training, may yield improved results on datasets where no single kernel is universally optimal.

Appendix A: Detailed Kernel Investigation

A.1 IBM HR: Kernel Comparison Across Types

Table A1 presents a detailed comparison of kernel methods on the IBM HR attrition dataset across all tested kernel types. The RBF kernel consistently outperformed the linear kernel, providing empirical evidence that the attrition decision manifold is nonlinear.

Model	Kernel	Accuracy	Precision (M)	Recall (M)	F1 (M)
KRR	Linear	0.5375	0.5192	0.5019	0.3727
KRR	RBF	0.7875	0.8073	0.7967	0.7867
Kernel SVM	Linear	0.7000	0.7011	0.7021	0.6998
Kernel SVM	RBF	0.7750	0.7986	0.7850	0.7737
Kernel KNN	Linear	0.6250	0.7190	0.6474	0.6000
Kernel KNN	RBF	0.6250	0.7190	0.6474	0.6000
KPCA+LogReg	RBF	0.5250	0.6751	0.5563	0.4473

Table A1: IBM HR kernel comparison across Linear and RBF kernels.

The performance gap between linear and RBF kernels is most pronounced for KRR, where accuracy improved from 53.8% (linear) to 78.8% (RBF), a gain of 25 percentage points. This substantial difference is explained by the nature of the attrition decision boundary: employees leave due to complex interactions between factors (e.g., high overtime combined with low job satisfaction and long commute distance), which create a nonlinear decision surface that linear kernels cannot represent. The RBF kernel, by computing similarity based on Euclidean distance in the original feature space and implicitly mapping to an infinite-dimensional feature space, successfully captures these multi-way interactions.

The Kernel SVM showed a smaller but consistent improvement from linear (70.0%) to RBF (77.5%). The SMO optimizer's convergence behavior differed substantially between kernels: the linear kernel required 36.3 seconds to train on 350 samples due to many passes needed to find the optimal separating hyperplane in the original space, while the RBF kernel converged in 0.6 seconds because the data becomes more separable after the implicit feature mapping.

Notably, Kernel KNN produced identical results for both linear and RBF kernels on this dataset (62.5% accuracy). This occurs because the kernel-induced distance $d^2(x,z) = k(x,x) + k(z,z) - 2k(x,z)$ can produce similar neighbor rankings when the local neighborhood structure is preserved across kernel transformations. The relatively poor performance of Kernel KNN compared to standard KNN (74.6%) suggests that the kernel-induced distance metric does not improve upon Euclidean distance for this dataset's manifold structure.

A.2 Mathematical Justification for Kernel Selection

The kernel function $k(x, x')$ defines a similarity measure in an implicit feature space. Following the course lecture (Slide 6), a kernel function satisfies $k(x, z) = \phi(x)^T \phi(z)$, where ϕ is the feature mapping. The choice of kernel determines the geometry of the decision boundary:

Linear kernel: $k(x, x') = x^T x' + c$ projects data into the original feature space (or a trivially extended version). This is appropriate when the class boundary is approximately linear, which was the case for certain complaint categories in the NYC 311 dataset where geographic coordinates alone provided sufficient separability.

Polynomial kernel: $k(x, x') = (\alpha x^T x' + c)^d$ maps data into a space of all monomials up to degree d . With $d=3$, this captures interactions between up to three features simultaneously. This is appropriate for data with known polynomial relationships, such as the interaction between property size, location, and amenities in the Airbnb dataset.

RBF (Gaussian) kernel: $k(x, x') = \exp(-\|x - x'\|^2 / (2 \sigma^2))$ maps data into an infinite-dimensional feature space. The parameter σ (equivalently $\gamma = 1/(2 \sigma^2)$) controls the locality of the similarity measure: small σ values make only very nearby points similar, while large σ values create broader similarity neighborhoods. The default $\sigma = \sqrt{d}$ provides a data-adaptive bandwidth that scales with dimensionality. The RBF kernel is appropriate when the decision boundary is nonlinear and localized, as observed in the IBM HR attrition prediction.

A.3 Kernel Matrix Properties

The kernel matrix K (Gram matrix) where $K_{ij} = k(x^i, x^j)$ must be symmetric and positive semi-definite for the kernel to correspond to a valid inner product in some feature space (Mercer's condition). All three implemented kernels satisfy this condition. The conditioning of the kernel matrix directly impacts the numerical stability of the dual solution $\alpha = (K + \lambda I)^{-1} y$. The regularization parameter λ plays a dual role: (1) it controls the complexity of the learned function, and (2) it ensures the matrix $(K + \lambda I)$ is invertible by guaranteeing all eigenvalues are at least λ . In our experiments, $\lambda = 1.0$ provided consistent numerical stability across all datasets.

Appendix B: Detailed Per-Class Results

B.1 IBM HR: Decision Tree Per-Class Breakdown

Table B1 presents the per-class performance of the best-performing model (Decision Tree) on the IBM HR dataset after SMOTE balancing.

Class	Precision	Recall	F1-Score	Support
0 (No Attrition)	0.8157	0.8455	0.8303	246
1 (Attrition)	0.8397	0.8089	0.8240	246
Macro Average	0.8277	0.8272	0.8272	492
Weighted Average	0.8277	0.8272	0.8272	492

Table B1: Decision Tree per-class metrics for IBM HR attrition prediction.

The Decision Tree achieved balanced performance across both classes after SMOTE balancing, with precision of 81.6% for non-attrition and 84.0% for attrition. The recall for the attrition class (80.9%) is particularly important in this application, as missing actual attrition cases has higher business cost than false positives. The confusion matrix shows 208 correct non-attrition predictions, 199 correct attrition predictions, 38 false positives (predicted attrition when actual non-attrition), and 47 false negatives (missed attrition cases).

B.2 NYC 311: Per-Class Analysis for Top Complaint Categories

Table B2 shows the per-class breakdown for the Logistic Regression classifier on the NYC 311 dataset, revealing the stark performance disparity between majority and minority classes.

Class ID	Complaint Type	Precision	Recall	F1	Support
0	Illegal Parking	0.5814	0.9740	0.7282	77
1	Noise - Residential	1.0000	1.0000	1.0000	33
2	Blocked Driveway	0.0000	0.0000	0.0000	23
3	Noise - Street	0.0000	0.0000	0.0000	22
4	Unsanitary Condition	0.5556	1.0000	0.7143	20
5	Street Condition	1.0000	1.0000	1.0000	19
6	Water System	1.0000	1.0000	1.0000	16
7	Abandoned Vehicle	0.0000	0.0000	0.0000	10
8	Plumbing	0.0000	0.0000	0.0000	8
9	Paint/Plaster	0.0000	0.0000	0.0000	8

Table B2: NYC 311 per-class metrics for Logistic Regression.

The results reveal a bimodal performance pattern: classes with distinct feature signatures (Noise - Residential, Street Condition, Water System) achieve perfect classification, while semantically overlapping classes (Blocked Driveway, Noise - Street, Abandoned Vehicle) achieve zero recall. This suggests that the available features (temporal, geographic, administrative) are sufficient to distinguish certain complaint types but inadequate for others. The Blocked Driveway complaints (class 2) were systematically misclassified as Illegal Parking (class 0) due to overlapping geographic patterns and similar temporal distributions.

B.3 Online Retail: Segment Classification Detail

Segment	Precision	Recall	F1-Score	Support	% of Total
High-Value	0.8421	0.9231	0.8807	52	26.4%
Loyal	0.7321	0.7069	0.7193	58	29.6%
At-Risk	0.7625	0.8133	0.7871	75	37.8%
Lost	0.4000	0.1538	0.2222	13	6.2%

Table B3: Decision Tree per-segment metrics for Online Retail customer classification.

The Decision Tree successfully identified High-Value (92.3% recall) and At-Risk (81.3% recall) customers but struggled with the Lost segment (15.4% recall). The Lost segment represents only 6.2% of customers, creating a severe class imbalance. Furthermore, the RFM-based segmentation creates hard boundaries between segments: a customer near the boundary between At-Risk and Lost can have nearly identical feature values but different labels, making classification inherently difficult at the margins.

B.4 Regression Performance Summary Across All Datasets

Dataset	Model	R2	RMSE	MAE	Target Mean	Target Std
Airbnb	Linear Regression	0.5070	147.31	87.12	206.72	209.80
Airbnb	KNN (k=5)	0.5083	147.11	86.46	206.72	209.80
Airbnb	Decision Tree	0.1590	192.40	96.03	206.72	209.80
Airbnb	KRR (RBF)	0.3078	185.93	97.50	205.91	223.47
NYC 311	Linear Regression	0.3339	5.16	3.53	4.30	6.32
NYC 311	KNN (k=5)	0.3984	4.90	3.07	4.30	6.32
NYC 311	Decision Tree	0.1547	5.81	3.37	4.30	6.32
NYC 311	KRR (RBF)	0.2089	4.84	3.07	3.59	5.44

Online Retail	Linear Regression	0.6014	0.5400	0.4160	5.91	0.86
Online Retail	KNN (k=5)	0.7106	0.4602	0.3645	5.91	0.86
Online Retail	Decision Tree	0.7820	0.3994	0.2553	5.91	0.86
Online Retail	KRR (RBF)	-0.2456	0.8798	0.4061	5.74	0.79

Table B4: Comprehensive regression results across all datasets and models.

The regression results reveal a clear pattern: the Online Retail dataset (log-transformed monetary) was the most predictable (best $R^2=0.782$), followed by Airbnb price ($R^2=0.508$) and NYC 311 resolution time ($R^2=0.398$). The predictability hierarchy reflects the inherent noise in each target variable. Customer spending is driven by measurable behavioral patterns (frequency, recency, product diversity) that are directly encoded in the features. Airbnb price depends on a combination of measurable attributes (location, size, amenities) and unmeasurable factors (listing quality, host responsiveness quality). Resolution time is the least predictable because it depends on agency workload and resource availability, factors entirely absent from the dataset.